

Diepe code-uitleg

De grafieken, regel voor regel

aquarium.js · languages.js · net.js · radar.js

Elke code-snipppet uitgelegd zodat je het zelf kunt aanpassen

Vervolg op "De Visdeurbel-datavisualisatie — stap voor stap".

Bestanden in `src/scripts/mitchell/charts/`

Zo lees je dit document

Per grafiek lopen we de code **van boven naar beneden** door in kleine blokjes. Onder elk blokje staat per regel wat hij doet — en waar nuttig een **Aanpassen-tip**.

- **Code-blokken** zijn de échte regels uit het bestand (donkere achtergrond).
- De **nummers** links in een blok verwijzen naar het regelnummer in het bronbestand.
- **Aanpassen** = een concrete knop waar je aan kunt draaien (kleur, snelheid, aantallen...).

Terugkerende begrippen — `const` / `let` = een waarde een naam geven (`const` = vast, `let` = mag wijzigen). `=>` = een "pijl-functie" (kort recept). `d3` = de tekenbibliotheek. `state` = de gedeelde data. `$('#id')` = zoek het HTML-element met dat id. Een volledige begrippenlijst staat achterin.

Beschrijvende namen. De code gebruikt uitgeschreven namen: `context` (het canvas-penseel), `pixelRatio` (scherm scherpte), `stageWidth/stageHeight` (afmetingen), `fishes` (de visjes), `velocityX/velocityY` (snelheid). Zo lees je vrijwel uit de naam al wat iets doet.

1 · aquarium.js CANVAS

src/scripts/mitchell/charts/aquarium.js

Tientallen visjes zwemmen rond op één `<canvas>`. Aantal per soort = hoe vaak gezien; grootte = gewicht. Klik = schrikken + rimpel, hover = naam & lengte.

1.1 — Imports (regel 1–4)

```
1 import { $, formatNumber, reduceMotion } from '../utils.js';
2 import { fishImagePath, hexToRgb01 } from '../fishImage.js';
3 import { showTooltip, hideTooltip } from '../tooltip.js';
4 import { state, lifecycle, raf } from '../state.js';
```

`$` = element opzoeken; `formatNumber` = getal met punten; `reduceMotion` = "animaties uit?".

`fishImagePath` = pad naar de vis-PNG; `hexToRgb01` = hex-kleur → 3 getallen 0–1 (voor het inkleuren).

`showTooltip/hideTooltip` = de gedeelde tooltip tonen/verbergen.

`state` = gedeelde data, `lifecycle` = opruimlijst, `raf` = animatieframe dat netjes te stoppen is.

1.2 — Start & canvas opzetten (regel 13–20)

```
13 export function initAquarium() {
14   const { TOTAL, visData } = state;
15   const { cleanups } = lifecycle;
16   const stage = $('#aquariumStage');
17   if (!stage) return;
18   stage.querySelectorAll('canvas, .aquarium-counter').forEach(n => n.remove());
19   const canvas = document.createElement('canvas');
20   stage.appendChild(canvas);
```

14 Haal `TOTAL` (totaal waarnemingen) en `visData` (de vissoorten) uit de gedeelde data.

15 Pak de opruimlijst, zodat we later onze timers/animaties kunnen afmelden.

16–17 Zoek het podium-`div`; bestaat het niet, stop dan meteen (voorkomt fouten).

18 Ruim een eventueel oud canvas/teller op (bij hertekenen na periodewissel).

19–20 Maak een nieuw `<canvas>` en hang het in het podium.

1.3 — Teller, tekencontext & scherpte (regel 21–35)

```
21 const counter = document.createElement('div');
22 counter.className = 'aquarium-counter';
23 counter.setAttribute('aria-live', 'polite');
24 stage.appendChild(counter);
25 const context = canvas.getContext('2d');
26 const pixelRatio = Math.min(window.devicePixelRatio || 1, 2);
27 let stageWidth = 0, stageHeight = 0;
28 function resize() {
29   const rect = stage.getBoundingClientRect();
30   stageWidth = rect.width; stageHeight = rect.height;
31   canvas.width = stageWidth * pixelRatio; canvas.height = stageHeight * pixelRatio;
32   context.setTransform(pixelRatio, 0, 0, pixelRatio, 0, 0);
33 }
34 resize();
35 const resizeObserver = new ResizeObserver(resize); resizeObserver.observe(stage);
```

21–24 Maak het teller-vakje. `aria-live="polite"` laat voorleessoftware de oplopende teller rustig melden.

25 `context` is het "penseel": alle tekenopdrachten lopen hierover.

26 `pixelRatio` = schermresolutie (retina = 2). Met `Math.min(..., 2)` kappen we op 2 af — hoger kost veel rekenkracht zonder zichtbaar verschil.

27 `stageWidth / stageHeight` in CSS-pixels; nog 0, worden in `resize()` gevuld.

28-33 `resize()`: meet het podium, zet het canvas op echte pixels (× `pixelRatio`) en schaal het penseel met `setTransform` zodat je in gewone pixels kunt tekenen (scherp, niet wazig).

34-35 Eén keer meten, en met een `ResizeObserver` opnieuw meten als het venster verandert.

Aanpassen. Wil je scherper op zeer hoge schermen? Verhoog de 2 in regel 26 (kost performance).

1.4 — Totaal & samenvatting (regel 37–46)

```
37 const total = TOTAL || visData.reduce((sum, v) => sum + v.count, 0) || 1;
38 const totalEl = $('#aquariumTotal'); if (totalEl) totalEl.textContent = formatNumber(total);
41 const sortedSpecies = visData.slice().sort((a, b) => (b.count || 0) - (a.count || 0));
42 const top3Names = sortedSpecies.slice(0, 3).map(s => s.naam).join(', ');
43-46 // zet samenvattingszin in #aquariumSummary
```

37 `total` = het totaal. Eerst `TOTAL`; is dat 0, tel dan alle `count`'s op (`reduce`); en `|| 1` voorkomt deling-door-nul later.

38 Toon het totaal in de kop (`#aquariumTotal`), netjes geformatteerd.

41 `slice()` maakt een **kopie** (anders sorteert je de gedeelde `visData`!), daarna sorteren op aantal, hoog→laag.

42 Pak de `top3Names` en plak ze met komma's aan elkaar voor de zin.

1.5 — Instelbare waarden (regel 48–58)

```
49 const FISH_COUNT = 80; // totaal aantal visjes
50 const SIZE_SCALE = 0.85; // algehele grootte
51 const SCARE_RADIUS = 35; // schrik-straal (in %)
52-53 const SCARE_PUSH_X/Y = 2.5/1.6; // hoe hard ze wegschieten
54 const CENTER_PULL = 0.00006; // trek naar het midden
55-56 const WANDER/FRICTION = 0.01/0.992; // dwalen / afremmen
57-58 const DRAW_ALPHA/COUNTER_MS = 0.92/3800;
```

Alle "draaiknoppen" van het aquarium staan hier bij elkaar, met uitleg-comment per regel. Verderop in de code worden ze bij naam gebruikt (bijv. `FRICTION`) i.p.v. losse getallen — zo verander je gedrag op één plek.

1.6 — Vis-plaatjes inkleuren: de cache (regel 60–74)

```
61-62 const spriteCache = {}, imageCache = {};
63 function loadImage(url) {
64   if (imageCache[url]) return imageCache[url]; // al geladen? hergebruik
65-68   const img = new Image(); img.src = url; imageCache[url] = img; return img;
70   function buildSprite(naam, color) {
71     const key = naam + color;
72     if (spriteCache[key]) return spriteCache[key]; // al ingekleurd? hergebruik
73     const img = loadImage(fishImagePath(naam));
74     if (!img.complete || !img.naturalWidth) return null; // nog niet binnen
```

61-62 Twee "kaartenbakken": `spriteCache` (ingekleurde resultaten) en `imageCache` (rauwe afbeeldingen). Cachen = werk maar één keer doen.

63-68 **loadImage**: is de afbeelding al geladen, geef 'm terug; anders start het laden en onthoud 'm.

70-72 **buildSprite**: de sleutel is `naam+color`; bestaat die al, klaar. Zo krijgt elke soort+kleur precies één ingekleurde versie.

74 Is de PNG nog niet binnen (`complete / naturalWidth`), geef `null` terug — de tekenlus probeert het dan volgende frame opnieuw.

1.7 — Pixel-voor-pixel inkleuren (regel 75–105)

```
75-76 const aspectRatio = ...; const spriteWidth = 64, spriteHeight = 64 * aspectRatio;
77-81 // teken de PNG op een mini-canvas 'spriteCanvas' (64px breed)
87   const whiteMix = 0.4, brightnessLift = 1.12;
88   const [red, green, blue] = hexToRgb01(color);
89-91 const tintR = red + (1 - red) * whiteMix; // kleur richting wit (lichter)
93-94 const imageData = spriteContext.getImageData(...); const pixels = imageData.data;
95   for (let i = 0; i < pixels.length; i += 4) {
96     if (pixels[i + 3] === 0) continue; // transparant → overslaan
97     const luminance = 0.299*pixels[i] + 0.587*pixels[i+1] + 0.114*pixels[i+2];
98-100 pixels[i] = Math.min(255, luminance * tintR * brightnessLift); // grijs × kleur
102-104 spriteContext.putImageData(imageData, ...); spriteCache[key] = spriteCanvas; return spriteCanvas;
```

75–76 Bereken de beeldverhouding en zet de sprite op 64px breed (hoogte volgt de verhouding).

87 `whiteMix` = hoeveel de kleur richting wit wordt gemengd (lichter); `brightnessLift` = extra helderheid. Hoger = lichtere visjes.

88–91 Zet de hex-kleur om naar drie getallen 0–1 en meng elke component richting wit: `kleur + (1-kleur)·whiteMix`.

93–94 Lees alle pixels uit. `pixels` is één lange rij: per pixel 4 waarden (rood, groen, blauw, doorzichtigheid).

95–96 Loop met stappen van 4 (één pixel per keer); een volledig doorzichtige pixel overslaan.

97 Bereken de **grijswaarde** `luminance` met de standaard ogen-weging (groen telt het zwaarst). Zo behouden we licht/donker = textuur.

98–100 Vermenigvuldig die grijswaarde met de doelkleur → de vis krijgt zijn kleur maar houdt zijn schaduwen. `Math.min(255,...)` voorkomt te-felle pixels.

102–104 Zet de bewerkte pixels terug, bewaar in de cache en geef de ingekleurde mini-canvas terug.

Aanpassen. Donkerder/feller visjes: verlaag `whiteMix` of `brightnessLift` (regel 87). Een andere kleur per soort regel je in `constants.js` (`color`), niet hier.

1.8 — Grootte op schaal & de steekproef van ~80 (regel 111–127)

```
111 const sizeForWeight = (kg) => Math.cbrt(kg) * SIZE_SCALE;
114 const fishes = [];
115 visData.forEach(species => {
116   const fishCountForSpecies = Math.max(1, Math.round((species.count / total) * FISH_COUNT));
117   for (let i = 0; i < fishCountForSpecies; i++) fishes.push({
118     x: Math.random()*100, y: Math.random()*100,
119     velocityX: (Math.random()-0.5)*0.4, velocityY: (Math.random()-0.5)*0.2,
120-121 color: species.color, naam: species.naam, size: sizeForWeight(species.weight),
124     lengthCm: Math.round((species.lengte || 30) * (0.82 + Math.random()*0.36)),
125     wigglePhase: Math.random()*Math.PI*2, visible: true,
126-127   });
```

111 **sizeForWeight**: grootte = $\sqrt[3]{\text{gewicht}} \times \text{SIZE_SCALE}$. Derdemachtswortel omdat massa met lengte³ groeit → juiste verhouding tussen soorten.

115–116 Voor elke soort: aantal visjes = aandeel in het totaal × `FISH_COUNT`, minimaal 1 (anders verdwijnt een zeldzame soort).

118 Startpositie willekeurig in 0–100 (we werken in **procenten** van het kijkglas, niet in pixels — handig bij resize).

119 Startsnellheid `velocityX / velocityY` licht willekeurig (de `-0.5` maakt 'm even vaak negatief als positief = beide richtingen).

120–121 Bewaar kleur, naam en de berekende grootte.

124 Lengte per individu: typische soort-lengte (`species.lengte`) $\pm$ ~18% toeval, voor de hover-tekst.

125 `wigglePhase` = een willekeurige start-fase voor de zwem-golf; `visible` = aan/uit via de filters.

Aanpassen. Meer/minder visjes: `FISH_COUNT` (regel 49). Algehele grootte: `SIZE_SCALE` (regel 50).

1.9 — Filter-chips per soort (regel 129–145)

```
133 visData.forEach(species => {
134-137 // maak een knop met de soort-naam en -kleur (via CSS-variabele --chip)
138   chip.addEventListener('click', () => {
139     const muted = chip.classList.toggle('muted');
140     chip.setAttribute('aria-pressed', String(!muted));
141     fishes.forEach(fish => { if (fish.naam === species.naam) fish.visible = !muted; });
142-143   });
```

134–137 Maak per soort een knopje (chip) met de naam en de soort-kleur.

139 `classList.toggle('muted')` zet de gedempte stijl aan/uit én geeft terug of hij nu aan staat.

140 Houd het toegankelijkheids-attribuut `aria-pressed` in sync.

141 Zet alle `fishes` van die soort op zichtbaar/onzichtbaar — de tekenlus slaat onzichtbare over.

1.10 — Klik = schrikken + rimpel (regel 147–170)

```
147-148 const toPercent = (event, rect) => [(event.clientX-rect.left)/rect.width*100, ...]; // muis → procent
151-153 canvas.addEventListener('click', (event) => { const rect = ...; const [clickX, clickY] = toPercent(event, rect);
154-156 fishes.forEach(fish => { const deltaX = fish.x-clickX, deltaY = fish.y-clickY; const distance = Math.sqrt(...);
158-160 if (distance >= SCARE_RADIUS) return; const strength = (SCARE_RADIUS-distance)/SCARE_RADIUS; const safeDistance = Math.max(0.1, distance);
161-162 fish.velocityX += (deltaX/safeDistance)*strength*SCARE_PUSH_X; fish.velocityY += ...*SCARE_PUSH_Y;
164-169 // maak een rimpel-element op de klikplek, verwijder na 950ms
```

147–148 `toPercent` rekent de muispositie om naar dezelfde procent-coördinaten als de visjes. Eén hulpje, gedeeld door klik én hover.

154–156 Bereken per vis de afstand tot de klik (`distance` via Pythagoras).

158 Te ver weg ($\geq$ `SCARE_RADIUS`)? Niet schrikken.

159–162 `strength` = sterkte (dichterbij = sterker). `deltaX/safeDistance` is de richting wég van de klik; `Math.max(0.1,...)` voorkomt delen door 0.

164–169 Maak een `<span class="aquarium-rip">` op de klikplek (de CSS doet de rimpel-animatie) en ruim 'm na 950ms op.

Aanpassen. Groter schrikbereik: `SCARE_RADIUS` (regel 51). Heftiger wegschieten: `SCARE_PUSH_X/Y` (regel 52–53).

1.11 — Hover = naam & lengte (regel 172–184)

```
173 const [pointerX, pointerY] = toPercent(event, canvas.getBoundingClientRect());
174-175 let closestFish = null, closestDistance = 4; // alleen binnen 4% van een visje
176-180 fishes.forEach(fish => { if (!fish.visible) return; const distance = Math.hypot(...); if (distance < closestDistance) { ... } });
```

```
181   if (closestFish) { ...; showTooltip(`~${closestFish.naam}...~${closestFish.lengthCm} cm...`, ...); }
182   else { ...; hideTooltip(); }
```

173–175 Reken de muispositie om en zoek het dichtstbijzijnde visje, maar alleen binnen 4 (procent) — anders geen tooltip.

176–180 Loop alle zichtbare visjes langs en onthoud de dichtstbijzijnde (`Math.hypot` = afstand).

181 Is er één dichtbij: cursor → pointer en toon de tooltip met **naam + ~lengte cm**.

182 Anders: cursor → kruisje en verberg de tooltip.

1.12 — De tekenlus `tick()` (regel 187–231)

```
188   context.clearRect(0, 0, stageWidth, stageHeight); // scherm leegvegen
189-191 // teken een zachte radiale gloed als achtergrond
192   fishes.forEach(fish => {
194-196 fish.x += fish.velocityX; fish.y += fish.velocityY + Math.sin(fish.wigglePhase)*0.04; fish.wigglePhase +=
0.08;
198-199 fish.velocityX += (50-fish.x)*CENTER_PULL; fish.velocityY += (50-fish.y)*CENTER_PULL;
200-201 fish.velocityX += (Math.random()-0.5)*WANDER; ... // dwalen
202-203 fish.velocityX *= FRICTION; fish.velocityY *= FRICTION; // wrijving
206-209 if (fish.x < -5) fish.x = 105; ... // rand → andere kant
211     if (!fish.visible) return;
212-213 const sprite = buildSprite(fish.naam, fish.color); if (!sprite) return;
216-217 const pixelX = (fish.x/100)*stageWidth, pixelY = (fish.y/100)*stageHeight; // procent → pixels
218-219 const angle = Math.atan2(fish.velocityY, fish.velocityX); const flip = Math.abs(angle) > Math.PI/2 ? -1 :
1;
222-228 context.save(); context.translate(pixelX, pixelY); context.rotate(...); context.scale(1, flip);
context.drawImage(sprite, ...); context.restore();
230   if (running) frameId = raf(tick); // volgende frame
```

188 Veeg het hele canvas leeg vóór een nieuw frame (anders smeren de visjes uit).

189–191 Teken een zachte turquoise gloed in het midden als achtergrond.

194–196 Beweeg: tel snelheid bij de positie op; `Math.sin(wigglePhase)` geeft een zachte verticale golf; de fase tikt door.

198–199 Zachte trek naar het midden (50%) via `CENTER_PULL`: hoe verder weg, hoe sterker het tikje terug.

200–201 Een vleugje toeval (`WANDER`) = "dwalen", zodat het natuurlijk oogt.

202–203 Wrijving: snelheid × `FRICTION` per frame, anders blijven ze versnellen.

206–209 Zwemt een vis voorbij de rand (−5 of 105), zet 'm aan de overkant terug (eindeloos kijkglas).

211–213 Onzichtbaar (uitgefilterd) of sprite nog niet geladen → dit frame overslaan.

216–219 Reken procent-positie om naar pixels; `angle` = kijkrichting uit de snelheid; `flip` spiegelt de vis als hij naar links zwemt.

222–228 `save/translate/rotate/scale/drawImage/restore` = teken de sprite op de juiste plek, gedraaid en evt. gespiegeld.

230 Vraag het volgende frame aan — maar alleen als `running` nog waar is (zo stopt de lus netjes).

Aanpassen. Rustiger zwemmen: verhoog `FRICTION` richting 0.98 (regel 56) of verklein `WANDER`. Sterkere samenscholing: verhoog `CENTER_PULL` (regel 54).

1.13 — Teller-animatie & aan/uit met de scroll (regel 232–254)

```
232-241 // animateCounter: telt in COUNTER_MS (3800ms) soepel op naar 'total' (ease-curve)
243   const visibilityObserver = new IntersectionObserver([entry]) => {
244-245     if (entry.isIntersecting) { if (!running && !reduceMotion()) { running = true; tick(); }
```

```
246-250    // teller één keer animeren; bij 'animaties uit' meteen eindstand
251    } else { running = false; cancelAnimationFrame(frameId); }
252-253 }, { threshold: 0.1 }); visibilityObserver.observe(stage);
254    cleanups.push(() => { running = false; cancelAnimationFrame(frameId); ...; visibilityObserver.disconnect();
resizeObserver.disconnect(); });
```

232-241 **animateCounter** laat het getal in 3,8s soepel oplopen met een ease-curve (eerst versnellen, dan afremmen).

244-245 De **IntersectionObserver** start de tekenlus zodra het aquarium in beeld is (en animaties aan staan).

246-250 De teller wordt maar één keer geanimeerd (**dataset.animated**); bij "animaties uit" springt hij meteen naar de eindstand.

251 Scrollt het weg: stop de lus en annuleer het frame (spaart de accu).

252-253 **threshold: 0.1** = "in beeld" zodra 10% zichtbaar is.

254 Opruimwerk: stop alles en koppel observers los. Belangrijk vóór een periodewissel.

2 · languages.js D3 · SVG

src/scripts/mitchell/charts/languages.js

Per taal een zwevend begroetingswoord; lettergrootte = aantal bezoekers. Een D3-*force-simulatie* duwt de woorden uit elkaar, daarna dobberen ze zacht.

2.1 — Instelbare waarden & afmetingen (regel 13–23)

```
14 const MAX_LANGUAGES = 24;           // hoeveel talen (drukste eerst)
15 const FONT_SIZE_MIN/MAX = 15/70;    // kleinste/grootste woord (px)
16-17 const PULL_TO_CENTER_X/Y = 0.04/0.06; // trek naar het midden
18-20 const REPEL_STRENGTH = -4; SETTLE_TICKS = 220; BOB_RADIUS = 5;
22-23 const STAGE_WIDTH = 900, STAGE_HEIGHT = 540, CENTER_X/Y = ...;
```

Alle draaiknoppen staan bovenaan het bestand (buiten de functie, want ze veranderen niet per render): hoeveel talen, de lettergrootte-range, de krachten van de simulatie en de dobber-straal.

22–23 Vast tekenvlak 900×540 (de SVG schaaft later mee); `CENTER_X/Y` = midden.

Aanpassen. Meer talen: `MAX_LANGUAGES` (14). Grotere woorden: `FONT_SIZE_MIN/MAX` (15). Heftiger dobberen: `BOB_RADIUS` (20).

2.2 — Start: data → woorden → tekenen (regel 25–61)

```
26-28 const { languagesData } = state; const stage = $('#langStage');
30-31 const words = buildWords(languagesData); if (!words.length) { ...'Geen taaldata.'...; return; }
32 const totalVisitors = d3.sum(words, word => word.visitors);
34-36 const svg = d3.select(stage).append('svg')...; const wordGroups = drawWords(svg, words);
attachTooltip(wordGroups, totalVisitors);
57-60 // zet de stat-zin "...X verschillende talen..." in #langStat
```

26–28 Pak de talen-data uit `state` en het podium.

30–31 `buildWords` (zie 2.4) zet de ruwe data om in tekenbare woorden; geen data → nette melding en stop.

32 `totalVisitors` = som van alle bezoekers (nodig voor de percentages in de tooltip).

34–36 Maak de SVG, teken de woorden (`drawWords`, 2.5) en koppel de tooltip (`attachTooltip`, 2.6).

2.3 — De force-simulatie & rust/dobberen (regel 38–55)

```
39 const simulation = d3.forceSimulation(words)
40-41 .force('x', d3.forceX(CENTER_X).strength(PULL_TO_CENTER_X)) .force('y', d3.forceY(CENTER_Y)...
42 .force('charge', d3.forceManyBody().strength(REPEL_STRENGTH))
43 .force('collide', d3.forceCollide(word => word.radius).strength(0.9))
44 .on('tick', () => positionWords(wordGroups));
46-50 if (reduceMotion()) { simulation.stop(); for (...SETTLE_TICKS...) simulation.tick(); positionWords(...); }
51-53 else simulation.on('end', () => startBobbing(words, wordGroups, cleanups));
55 cleanups.push(() => simulation.stop());
```

39 Start een **force-simulatie**: een nep-natuurkunde die elk woord een positie geeft.

40–41 `forceX` / `forceY` trekken zacht naar het midden (sterkte uit `PULL_TO_CENTER_X/Y`).

42 `charge` met negatieve sterkte (`REPEL_STRENGTH`) = woorden stoten elkaar licht af.

43 `collide` met straal `word.radius` = harde grens die overlap voorkomt.

44 Bij elke **tick** (rekenstap) verplaats je elke groep via `positionWords`.

46–50 **Animaties uit?** Reken in één keer `SETTLE_TICKS` stappen door en zet de woorden meteen neer.

51–53 Anders: zodra de simulatie tot rust komt ('end') start het zachte dobberen (2.7).

Aanpassen. Strakker op een kluitje: verhoog `PULL_TO_CENTER_X/Y`. Meer lucht tussen woorden: maak `REPEL_STRENGTH` negatiever.

2.4 — `buildWords()`: data → tekenbare woorden (regel 64–86)

```
65-66 const topLanguages = (languagesData || []).slice(0, MAX_LANGUAGES); if (!length) return [];
68   const maxVisitors = d3.max(topLanguages, lang => lang.n);
70   const fontSizeScale = d3.scaleSqrt().domain([0, maxVisitors]).range([FONT_SIZE_MIN, FONT_SIZE_MAX]);
72   const palette = [COLORS.green, COLORS.greenMid, COLORS.teal, COLORS.pink, COLORS.goldDeep];
74-76 return topLanguages.map((lang, i) => { const [greeting, name] = GREETINGS[lang.code] || [...]; const fontSize
= fontSizeScale(lang.n);
77-84 return { code, visitors: lang.n, greeting, name, fontSize, color: palette[i % ...], radius: ..., x: ..., y: ... };
});
```

65–66 Neem de **top** `MAX_LANGUAGES` talen (drukste eerst); geen data → lege lijst.

68–70 **fontSizeScale**: zet bezoekers om naar lettergrootte. `scaleSqrt` = wortelschaal, zodat de **oppervlakte** eerlijk meeschaalt.

72 Vijf kleuren om uit te kiezen (bewust geen paars: dat valt weg op de paarse sectie).

75 Zoek de begroeting + nette naam op in `GREETINGS`; onbekend → leeg woord + landcode in hoofdletters.

77–84 Per taal een object: `greeting` (het woord), `visitors`, `fontSize`, `color`, een `radius` (botsstraal, breder bij langere/grotere woorden) en een willekeurige startplek rond het midden.

2.5 — `drawWords()` & `positionWords()` (regel 89–113)

```
90-92 const wordGroups = svg.selectAll('g.lang-word').data(words).join('g')...attr('aria-label', word =>
` ${word.name}: ... bezoekers `);
94-98 wordGroups.append('text')...attr('font-size', word => word.fontSize)...text(word => word.greeting);
101-105 wordGroups.filter(word => word.fontSize > 30).append('text')...text(word => word.name);
111-112 // positionWords: wordGroups.attr('transform', word => `translate(${word.x},${word.y})`)
```

90–92 **Data-join**: koppel `words` aan `<g>`-groepen (één per taal). `aria-label` = "Japans: 1.234 bezoekers" voor voorleessoftware.

94–98 Voeg het grote begroetingswoord toe; `font-size` en kleur komen per taal uit de data.

101–105 Alleen bij grotere woorden (`fontSize > 30`) een klein taalnaam-label eronder — anders wordt het te druk.

111–112 **positionWords** verplaatst elke groep naar zijn huidige `x,y`. Dit roept zowel de simulatie-tick als het dobberen aan.

2.6 — `attachTooltip()` (regel 116–126)

```
117   const percentage = word => (word.visitors / totalVisitors * 100).toFixed(1);
118-120 wordGroups.on('mouseenter mousemove', (event, word) => showTooltip(`... ${word.name}...
${formatNumber(word.visitors)} bezoekers · ${percentage(word)}%`, ...))
121   .on('mouseleave blur', () => hideTooltip())
122-124   .on('focus', (event, word) => { ... showTooltip(..., box.left + box.width/2, box.top); });
```

117 **percentage**: aandeel van deze taal in het totaal, op 1 decimaal.

118–120 Bij hover: tooltip met naam, aantal en percentage.

121 Muis eraf of focus weg → tooltip verbergen.

122–124 Bij toetsenbord-focus de tooltip boven het woord plaatsen (`getBoundingClientRect` geeft de schermpositie).

2.7 — `startBobbing()`: zacht dobberen (regel 129–146)

```
130 words.forEach(word => { word.baseX = word.x; word.baseY = word.y; word.phase = Math.random()*Math.PI*2; });
134-135 const bobFrame = () => { const elapsedSeconds = (performance.now() - startTime) / 1000;
136-138 words.forEach(word => { word.x = word.baseX + Math.sin(elapsedSeconds*0.5 + word.phase)*BOB_RADIUS;
word.y = ...Math.cos(...)...; });
140-145 positionWords(wordGroups); bobFrameId = raf(bobFrame); }; ... cleanups.push(() =>
cancelAnimationFrame(bobFrameId));
```

130 Bewaar de rustplek (`baseX/baseY`) en geef elk woord een willekeurige `phase` (zodat ze niet synchroon deinen).

134-138 **bobFrame**: verplaats elk woord met een trage sinus/cosinus rond zijn rustplek (uitslag = `BOB_RADIUS`) = zacht dobberen.

140-145 Teken de nieuwe posities en vraag het volgende frame; bij opruimen het frame annuleren.

3 · net.js D3 · CIRCLE PACKING

src/scripts/mitchell/charts/net.js

Alle soorten als bellen in een net. Belgrootte = **aantal, gewicht/vis** of **biomassa** (aantal × gewicht). Wisselen laat de bellen soepel "morphen".

3.1 — Instelbare waarden & de drie weergaven (regel 13–36)

```
14-19 const STAGE_WIDTH/HEIGHT, NET_BOTTOM, BUBBLE_PADDING, FALL_FROM_OFFSET, FISH_SIZE_FACTOR/MAX,
    LABEL_MIN_RADIUS;
22-26 const valueForStat = { count: species => species.count, weight: species => species.weight, biomass: species
    => species.count * species.weight };
27-31 const summaryForStat = { ... }; // tekst per weergave (info-regel)
32-36 const explanationForStat = { ... }; // uitleg-zin onderaan
```

14-19 De draaiknoppen: tekenvlak, hoe ver het net hangt (`NET_BOTTOM`), ruimte tussen bellen (`BUBBLE_PADDING`), invalhoogte, vis-grootte en de drempel waaronder het label verdwijnt.

22-26 **valueForStat**: drie recepten voor de belgrootte. `biomass = count × weight`. Dit is het hart van de grafiek.

27-31 **summaryForStat**: de tekst in de info-regel per weergave (bijv. "... kg biomassa (aantal × gewicht)").

32-36 **explanationForStat**: korte uitleg-zin die onder de grafiek verschijnt.

3.2 — Opzet (regel 38–48)

```
39-43 const { visData } = state; const stageEl = $('#netStage'); const infoEl = $('#netInfo'); const svg =
    d3.select(stageEl).append('svg')...;
45-48 drawNet(svg); const bubbleLayer = svg.append('g')...; const defs = svg.append('defs'); let currentStat =
    'biomass';
```

39-43 Pak data, podium en info-regel; maak de SVG.

45 **drawNet** (zie 3.8) tekent het zakkende net.

46-48 Een laag voor de bellen, een `defs` voor de kleurverlopen, en de startweergave `'biomass'`.

3.3 — createBubble(): de inhoud van een bel (regel 51–66)

```
52-56 const gradientId = `bubGrad-...`; const gradient = defs.append('radialGradient')...; // licht → soort-kleur
58-59 group.append('circle').attr('class', 'bub-main').attr('r', bubble.r).attr('fill', `url(#${gradientId})`)...
60-62 group.append('circle').attr('class', 'bub-shine')... // glansplekje
63 group.append('g').attr('class', 'bub-fish')... // plek voor het vis-plaatje
64-65 group.append('text').attr('class', 'bub-label')... // het naam-label
```

52-56 Maak een eigen radiaal kleurverloop (lichte kern → soort-kleur rand) met een uniek `id`.

58-59 De hoofdcirkel, gevuld met dat kleurverloop. `bubble.r` = de door packing berekende straal.

60-62 Een klein lichter cirkeltje als "glans" linksboven.

63 Een lege groep waar straks het ingekleurde vis-plaatje in komt.

64-65 Het tekst-label met de soortnaam.

3.4 — updateBubble(): plaatje, info & soepel groeien (regel 70–88)

```
71-72 const fishSize = Math.min(bubble.r * FISH_SIZE_FACTOR, FISH_SIZE_MAX); const tintFilterId =
    ensureTintFilter(svg, bubble.data.color);
73-75 group.select('.bub-fish').html(`<image href="#" filter="url(#${tintFilterId})"/>`);
78-80 const showInfo = () => { infoEl.textContent = `${bubble.data.naam} – ${summaryForStat[stat]
    (bubble.data)}.`; }; group.on('click', showInfo).on('mouseenter', showInfo).on('keydown', ...);
```

```
82-87 group.select('.bub-main').transition().duration(800 * motionScale).attr('r', bubble.r); ... label: opacity =
bubble.r > LABEL_MIN_RADIUS ? 0.92 : 0;
```

71-72 Plaatjesgrootte $\sim$ `FISH_SIZE_FACTOR` $\times$ de straal (max `FISH_SIZE_MAX`). `ensureTintFilter` levert het kleur-filter (uit `fishImage.js`).

73-75 Zet het vis-plaatje in de bel, ingekleurd via dat filter.

78-80 **showInfo**: schrijf de cijfers van deze soort in de info-regel — bij klik, hover én Enter/Spatie (toetsenbord).

82-87 Animeer (`transition`) de cirkel, de glans en het label naar hun nieuwe grootte. `motionScale` is 0 bij "animaties uit" (dan meteen, geen overgang). Het label vervaagt als de bel kleiner is dan `LABEL_MIN_RADIUS`.

3.5 — `renderBubbles()`: data-join enter/update/exit (regel 91–115)

```
92-95 const packedBubbles = packBubbles(stat); const existingBubbles = bubbleLayer.selectAll('.net-
bubble').data(packedBubbles, bubble => bubble.data.naam);
98-99 existingBubbles.exit().transition().scale(0).remove(); // verdwenen → krimpen
102-105 const enteringBubbles = existingBubbles.enter().append('g').scale(0); enteringBubbles.each(... createBubble
...); // nieuw, hoog & klein
108 enteringBubbles.merge(existingBubbles).each(... updateBubble ...); // nieuw +
bestaand bijwerken
110-114 enteringBubbles.transition().scale(1); existingBubbles.transition().scale(1); // naar eindplek
animeren
```

92-95 Reken de posities uit (`packBubbles`, 3.6) en koppel ze met de **naam als sleutel**. Zo weet D3 welke bel bij welke soort hoort, ook na een wissel.

98-99 **exit** = soorten die er niet meer zijn → laat ze naar `scale(0)` krimpen en verwijder.

102-105 **enter** = nieuwe bellen → maak een groep, klein en `FALL_FROM_OFFSET` hoger (ze "vallen" straks in het net), en bouw de inhoud met `createBubble`.

108 `enter.merge(existing)` = nieuwe én bestaande bellen samen bijwerken met `updateBubble`.

110-114 Animeer iedereen naar zijn eindplek en volle grootte (`scale(1)`) — hierdoor "morphen" de bellen bij een wissel.

Waarom dit slim is. Door de naam als sleutel te gebruiken, blijft elke bel "dezelfde" tussen weergaven en schuift hij netjes naar zijn nieuwe grootte/plek — in plaats van weg te knippen en opnieuw te verschijnen.

3.6 — `packBubbles()`: circle packing (regel 118–123)

```
119 const bubbleData = visData.map(species => ({ ...species, value: valueForStat[stat](species) }));
120 const packLayout = d3.pack().size([STAGE_WIDTH-40, STAGE_HEIGHT-100]).padding(BUBBLE_PADDING);
121-122 const hierarchyRoot = d3.hierarchy({ children: bubbleData }).sum(species => species.value); return
packLayout(hierarchyRoot).leaves();
```

119 Geef elke soort een `value` volgens de gekozen weergave (de `...species` kopieert de bestaande velden).

120 `d3.pack` = de circle-packing-machine; `size` = beschikbaar vlak, `padding` = ruimte tussen bellen.

121-122 `d3.hierarchy(...).sum(...)` sommeert de `value`'s; `packLayout(...).leaves()` rekent posities + stralen uit en geeft de losse bellen terug.

3.7 — De schakelknoppen (regel 125–145)

```
125-126 renderBubbles(currentStat); infoEl.textContent = explanationForStat[currentStat]; // eerste render
130-134 const updateActiveToggle = () => $$('.net-toggle-btn').forEach(button => { ... markeer actief ... });
135-137 $$('.net-toggle-btn').forEach(button => { button.onclick = () => { if (button.dataset.stat ===
currentStat) return;
138-141 currentStat = button.dataset.stat; updateActiveToggle(); renderBubbles(currentStat); infoEl.textContent
```

```
= ...; }); });
145   cleanups.push(() => { $$('.net-toggle-btn').forEach(button => { button.onclick = null; }); });
```

125–126 Teken de eerste keer en zet de uitleg-zin.

130–134 `updateActiveToggle` markeert de actieve knop (class + `aria-selected`).

136 `button.onclick = ...` (i.p.v. `addEventListener`): zo overschrijf je bij hertekenen de oude handler i.p.v. er steeds één bij te stapelen.

137–141 Klik op de al-actieve knop? Niets. Anders: onthoud de nieuwe weergave, markeer 'm en teken opnieuw → de bellen morphen.

145 Bij opruimen de klik-handlers losmaken (geen geheugenlek).

Aanpassen. Andere startweergave: `'biomass'` (regel 48). Meer ruimte tussen bellen: `BUBBLE_PADDING` (16). Drempel waaronder het label verdwijnt: `LABEL_MIN_RADIUS` (19).

3.8 — `drawNet()`: het zakkende net (regel 149–169)

```
152-156 for (... i ≤ 16 ...) netGroup.append('path')...; // 17 verticale "touwen" (golvend via Math.sin)
157-161 for (... j ≤ 8 ...) netGroup.append('path')...; // 9 horizontale touwen, licht doorhangend (Q-boog)
166-168 netGroup.attr('transform', `translate(0, -${FALL_FROM_OFFSET})`).transition()...
attr('transform', 'translate(0,0)');
```

152–156 Teken 17 verticale touwen als licht golvende paden (de `Math.sin(i)` geeft de slinger).

157–161 Teken 9 horizontale touwen, elk iets doorhangend (`Q` = een gebogen lijn).

166–168 Begin het net `FALL_FROM_OFFSET` hoger en laat het met een `transition` in beeld zakken (overgeslagen bij "animaties uit").

4 · radar.js D3 · SVG · ANIMATIE

src/scripts/mitchell/charts/radar.js

Een sonar: elke soort is een "ping", afstand tot het midden = hoe vaak gezien. Een draaiende straal licht de pings op (naglooi). Een tijdslijder scrubt door dag/week/maand.

4.1 — Instelbare waarden & twee hulpjes (regel 16–35)

```
16-17 const WIDTH = 620, HEIGHT = 620, CENTER_X/Y = ..., RADIUS = 270;
18-20 const INNER_DISTANCE = 70, OUTER_DISTANCE = RADIUS-55, ABSENT_DISTANCE = OUTER_DISTANCE+10;
21-29 const MIN_GAP, JITTER_RANGE, PLACE_ATTEMPTS, SWEEP_SPEED(_REDUCED), REVEAL_ARC, MIN_GLOW, GLOW_MS,
SCRUB_MS;
32 const polarToPoint = (angle, distance) => [CENTER_X + Math.cos(angle)*distance, CENTER_Y +
Math.sin(angle)*distance];
34-35 const makeDistanceScale = (maxCount) => d3.scaleSqrt().domain([0, maxCount]).range([OUTER_DISTANCE,
INNER_DISTANCE]).clamp(true);
```

16–20 Vierkant vlak 620×620; **RADIUS** = buitenste ring; **INNER_DISTANCE** = dichtst bij het midden, **OUTER_DISTANCE** = bij de rand, **ABSENT_DISTANCE** = nét buiten de rand (0 waarnemingen).

21–29 Alle gevoels-knoppen: minimale tussenruimte, jitter, plaatsings-pogingen, draaisnelheid, naglooi-duur, laagste helderheid.

32 **polarToPoint**: zet een hoek + afstand om naar een `[x, y]` op het scherm.

34–35 **makeDistanceScale**: zet een aantal om naar een afstand. Let op de **omgekeerde** range `[OUTER, INNER]`: 0 → buiten, `maxCount` → binnen. `clamp(true)` houdt het binnen de grenzen.

Aanpassen. Sneller draaien: **SWEEP_SPEED** (24). Langer naglinsteren: **GLOW_MS** (28). Pings nooit helemaal kwijt: **MIN_GLOW** (27) hoger.

4.2 — Start: decor, pings & tijdvakken (regel 37–57)

```
38-40 const { visData, weekHours, weekDayLabels, weekDays, currentPeriod } = state; const svg =
d3.select($('#radarStage'))...;
42-43 drawDecor(svg); const sweep = drawSweep(svg);
46-49 const maxCount = Math.max(...visData.map(v => v.count || 0), 1); const placementScale =
makeDistanceScale(maxCount); const distanceForCount = (count, jitter, scale=...) => count > 0 ? scale(count)+jitter
: ABSENT_DISTANCE;
51-53 const pings = placePings(visData, distanceForCount); const timeBuckets = buildTimeBuckets(...); const
totalObservations = pings.reduce(...);
55-57 const pingsGroup = svg.append('g'); pings.forEach(ping => drawPing(ping, pings, pingsGroup, detailPanel,
svg));
```

38–40 Haal alles uit `state`: soorten, uren-per-dag, daglabels/datums en de actieve periode (die bepaalt straks de tijdvakken).

42–43 Teken het decor (4.4) en de sweep-wig (4.5).

46–49 `maxCount` = drukste soort; **distanceForCount** geeft de afstand voor een aantal (0 → net buiten de rand).

51–53 Plaats de pings (4.6), bouw de tijdvakken (4.10) en tel het totaal aantal waarnemingen.

55–57 Teken elke ping (4.7).

4.3 — De tijdslijder & scrubben `applyTime()` (regel 59–92)

```
60-62 const sliderLabel = makeSliderLabel(); const slider = makeSlider(timeBuckets); mountScrubber(sliderLabel,
slider, cleanups);
65-69 function applyTime(bucketIndex) { const isWholePeriod = bucketIndex === 0; sliderLabel.textContent =
```



```
isWholePeriod ? `Hele ...` : `${timeBuckets.labels[bucketIndex-1]}...`;
73-74 const countPerPing = pings.map(ping => isWholePeriod ? ping.total :
timeBuckets.perSpecies.get(ping.speciesIndex)?.[bucketIndex-1] ?? 0); const distanceScale =
makeDistanceScale(Math.max(...countPerPing, 1));
76-82 pings.forEach((ping, index) => { const count = countPerPing[index]; const distance = count > 0 ?
distanceScale(count)+ping.jitter : ABSENT_DISTANCE; [ping.x, ping.y] = polarToPoint(ping.angle, distance);
ping.elem.transition()...; });
83-85 ping.labelTextEl.text(`${ping.naam} · ${formatNumber(count)}`); ... if (ping.selected) ping.showDetail();
88-92 slider.addEventListener('input', () => applyTime(+slider.value)); applyTime(0); drawSummary(...);
startSweep(sweep, pings, cleanups);
```

60-62 Bouw de slider-bouwstenen (4.9) en plaats de scrubber ónder de radar.

66-69 `bucketIndex 0` = hele periode; anders een tijdvak. Zet de labeltekst boven de slider.

73 Haal het aantal per soort op voor deze stand (totaal, óf uit de verdeling van dit tijdvak).

74 **Sleutel-stap:** maak een **nieuwe** schaal genormaliseerd op de drukste soort van *dit* tijdvak. Daardoor staan de vissen altijd ten opzichte van elkáár.

76-82 Bereken per ping de nieuwe afstand (met de vaste `jitter`) en daaruit `x,y` — de hoek blijft gelijk, dus de ping beweegt alleen naar binnen/buiten; animeer er soepel naartoe.

83-85 Werk het label-aantal bij en ververs het detailpaneel als deze ping geselecteerd is.

88-92 Koppel de slider aan `applyTime`, zet 'm op stand 0, teken de samenvatting (4.8) en start de sweep (4.8).

Aanpassen. Absolute afstanden (vergelijkbaar tussen tijdvakken) i.p.v. relatief? Gebruik in regel 74 weer `maxCount` i.p.v. de tijdvak-max. Tragere overgang: `SCRUB_MS` (29).

4.4 — `drawDecor()`: schijf, ringen & dradenkruis (regel 96–117)

```
98-99 svg.append('circle')...attr('r', RADIUS).attr('fill', 'rgba(1,70,60,0.03)'); // lichte vulling
102-108 for (let ring = 1; ring <= 4; ring++) { const isOuterRing = ring === 4; svg.append('circle')...attr('r',
RADIUS*ring/4)...(buitenrand solide, rest gestippeld) }
112-116 const crosshairColor = ...; svg.append('line')... (horizontaal) ... svg.append('line')... (verticaal) //
dradenkruis
```

98-99 Een heel lichte schijf-vulling zodat de radar zich aftekent tegen de achtergrond.

102-108 Teken 4 ringen op straal `R/4`, `R/2`, `3R/4`, `R`. `isOuterRing` = de buitenste: donkerder/dikker en solide; de rest lichter en gestippeld (`stroke-dasharray`).

112-116 Twee lijnen (horizontaal + verticaal) door het midden vormen het dradenkruis.

4.5 — `drawSweep()`: de draaiende wig (regel 120–133)

```
121-124 const gradient = svg.append('defs').append('linearGradient')...; // transparant → groen
126-129 const [endX, endY] = polarToPoint(-Math.PI/4, RADIUS); const sweep = svg.append('path').attr('d', `M
${CENTER_X} ${CENTER_Y} L ... A ...`); // taartpunt van 45°
131-132 svg.append('circle')...attr('r', 5)...; return sweep; // stip in het midden
```

121-124 Een lineair kleurverloop: doorzichtig aan de ene kant, groen aan de andere — geeft de sweep zijn "veeg".

126-129 De sweep is een taartpunt-pad (`M` = ga naar, `L` = lijn, `A` = boog) van 45°. We geven 'm terug om straks te draaien.

131-132 Een klein stipje markeert het exacte midden.

4.6 — `placePings()` & `pickFreeSpot()`: zonder overlap (regel 136–164)

```
137-138 const seed = mulberry32(42); const pings = []; // vaste seed → zelfde plaatsing
139-142 visData.forEach((species, speciesIndex) => { const jitter = (seed()-0.5)*JITTER_RANGE; const distance =
```



```
distanceForCount(species.count, jitter); const freeSpot = pickFreeSpot(distance, pings, seed);
143-147 pings.push({ ...species, ...freeSpot, speciesIndex, jitter, total: ..., current: ..., litAt: -1e9, hover:
false, selected: false }); });
156-161 for (let attempt = 0; attempt < PLACE_ATTEMPTS; attempt++) { const angle = seed()*Math.PI*2; ... const
nearestDistance = ...; if (nearestDistance >= MIN_GAP) return { angle, x, y }; ... }
```

137-138 Een *seeded* toevalsgenerator (startwaarde 42) → elke render dezelfde plaatsing.

140 Een vaste `jitter` per ping (zodat hij bij scrubben niet "rondspringt", alleen radiaal beweegt).

141-142 De startafstand op basis van het totaal-aantal, plus een vrije plek (`pickFreeSpot`).

143-147 Bewaar de ping: alle soort-velden (`...species`), de hoek/positie (`...freeSpot`), de index, jitter, `total / current` en de animatie-vlaggen (`litAt` = wanneer laatst opgelicht, `hover`, `selected`).

156-161 **Rejection sampling**: probeer tot `PLACE_ATTEMPTS` × een willekeurige hoek; is de afstand tot de dichtstbijzijnde ping groot genoeg (`MIN_GAP`), neem die. Lukt het niet, neem de beste poging.

4.7 — `drawPing()`: tekenen + info op hover (regel 167-215)

```
168-172 const pingGroup = pingsGroup.append('g')...attr('opacity', 0)...; // start onzichtbaar
174-175 pingGroup.append('circle')...r 18 (halo); ...r 8 (kern)
177-181 // ingekleurd vis-plaatje via ensureTintFilter
184-190 const labelOnRight = ...; ping.labelTextEl = pingGroup.append('text')...text(`${ping.naam} ·
${formatNumber(ping.count)}`);
192-199 const showDetail = () => { ... detailPanel.innerHTML = `...piekmaand, diepte, gewicht...`;
detailPanel.classList.add('visible'); };
200-211 const hideDetail = () => { ... }; pingGroup.on('mouseenter', ...showDetail).on('mouseleave',
hideDetail).on('focus', showDetail).on('blur', hideDetail);
```

168-172 Een groep per ping, **start onzichtbaar** (`opacity 0`) — de sweep onthult 'm straks.

174-175 Twee cirkels: een grote vage halo (`r=18`) en een vollere kern (`r=8`) in de soort-kleur.

177-181 Het ingekleurde vis-plaatje, met het tint-filter uit `fishImage.js`.

184-190 Het label "naam · aantal". `labelOnRight` bepaalt of het rechts of links van de vis staat (zodat het niet buiten beeld valt). We bewaren het in `ping.labelTextEl` om het bij scrubben te updaten.

192-199 **showDetail**: maak deze ping de geselecteerde (de rest niet) en vul het detailpaneel met naam, huidig aantal, piekmaand, diepte en gewicht.

200-211 **hideDetail** zet alles terug. Koppel: muis erop / focus → tonen; muis eraf / blur → verbergen.

Aanpassen (info). Welke regels in het paneel staan, zit in `showDetail` (regel 197). Wil je info laten staan i.p.v. verdwijnen: haal het verbergen uit `hideDetail`.

4.8 — `startSweep()` draaien & `glowOpacity()` naglooi (regel 227-259)

```
230-232 function tick(now) { sweepAngle += reduceMotion() ? SWEEP_SPEED_REduced : SWEEP_SPEED;
sweep.attr('transform', `rotate(...)`);
234-237 pings.forEach(ping => { const angleBehindSweep = (ping.angle - sweepAngle + Math.PI*4) % (Math.PI*2); if
(angleBehindSweep < REVEAL_ARC) ping.litAt = now; ping.elem.attr('opacity', glowOpacity(ping, now)); });
243-248 // IntersectionObserver: start tick in beeld, stop buiten beeld
252-258 function glowOpacity(ping, now) { if (reduceMotion()) return 1; if (ping.hover || ping.selected) return 1;
if (ping.litAt < 0) return 0; const age = (now - ping.litAt) / GLOW_MS; const glow = age >= 1 ? 0 : (1 - age) * (1 - age);
return MIN_GLOW + (1 - MIN_GLOW) * glow; }
```

230-232 Elke frame draait de sweep een stukje verder (`SWEEP_SPEED`, sneller bij "animaties uit") en past de rotatie toe.

234-237 `angleBehindSweep` = hoekverschil tussen ping en sweep. Veegt de sweep er net overheen (`< REVEAL_ARC`), onthoud dan "nu opgelicht" (`ping.litAt = now`), en pas de helderheid toe.

243–248 Start de lus als de radar in beeld is, stop 'm als hij weg scrolt (accu sparen).

252–258 **glowOpacity**: animaties uit → vol; hover/geselecteerd → vol; nog nooit gezien → onzichtbaar; anders een **uitdovende gloed** (kwadratisch) die nooit onder `MIN_GLOW` zakt.

4.9 — Tijdslijder-bouwstenen (regel 262–292)

```
262-266 function makeSliderLabel() { ... // het tekstlabel boven de slider
267-278 function makeSlider(timeBuckets) { ... range van 0 t/m timeBuckets.labels.length, stap 1, start 0 ...
279-292 function mountScrubber(label, slider, cleanups) { ... $('#radarStage').insertAdjacentElement('afterend',
scrubberBox); cleanups.push(() => scrubberBox.remove()); }
```

262–266 **makeSliderLabel**: het tekstlabel dat boven de slider de huidige stand toont.

267–278 **makeSlider**: een schuifbalk (`input type=range`) van 0 t/m het aantal tijdvakken; stap 1; start op 0 (= hele periode).

279–292 **mountScrubber**: maak een doosje (gecentreerd, lichte achtergrond) en plaats het **direct ná** de radar — dus eronder, zonder de radar te bedekken. Bij opruimen weghalen.

4.10 — buildTimeBuckets() & groupDays() (regel 295–352)

```
299-304 // dayTotals: tel per dag alle 24 uren uit weekHours op (de echte "drukte-vorm")
306   const groups = groupDays(currentPeriod, dayCount, weekDayLabels, weekDays);
309-311 const bucketTotals = groups.map(... sommeer dayTotals ...); const weights = bucketTotals.map(v =>
v/bucketTotalsSum);
314-317 visData.forEach((species, speciesIndex) => { perSpecies.set(speciesIndex,
distributeOverBuckets(species.count||0, weights, mulberry32(1000+speciesIndex*7))); });
325-327 if (period === 'week') return ... elke dag = 1 tijdvak ...
329-341 if (period === 'jaar') { ... groepeer per kalendermaand op basis van de datum ... }
344-351 // anders (maand): hak de dagen in blokken van 7 → "Week 1...5"
```

299–304 Maak **dayTotals**: per dag de som van de 24 uur-waarden uit **weekHours**.

306 **groupDays** bepaalt de tijdvakken op basis van de periode.

309–311 **bucketTotals** = totale drukte per tijdvak; **weights** = die drukte als fractie van het geheel (samen 1).

314–317 Verdeel per soort het totaal over de tijdvakken (`distributeOverBuckets` , 4.11), met een eigen seed per soort.

325–327 **week**: elke dag wordt een eigen tijdvak (label = de datum).

329–341 **jaar**: groepeer de dagen per kalendermaand (op basis van de datum); label = maandnaam.

344–351 **maand**: hak de dagen in blokken van 7 → "Week 1...5", met de datumreeks als sub-label.

4.11 — distributeOverBuckets(): kloppend tot het totaal (regel 356–370)

```
357-360 const raw = weights.map(weight => Math.max(0.0001, weight*(0.5+seed()))); ... const exact = raw.map(x =>
(x/rawSum)*total); const rounded = exact.map(Math.round);
362-368 let diff = total - ...; const order = ... (grootste rest eerst) ...; for (... diff !== 0 ...) { rounded[i] += step;
diff -= step; }
```

357 **raw** = de drukte-vorm (**weights**) met een soort-eigen variatie (`0.5 + seed()`) eroverheen, zodat niet alle soorten identiek bewegen.

359–360 Schaal terug zodat de som = het soort-totaal, en rond af op hele vissen.

362–368 Afronden verandert de som; het verschil (**diff**) wordt over de tijdvakken met de grootste afrondings-rest verdeeld zodat de som **exact** klopt.

Waarom afgeleid? Er is geen kant-en-klare data "soort × tijdvak". We gebruiken de échte drukte per dag als vorm en verdelen elk soort-totaal daarover, met een vleugje variatie. Realistisch én altijd kloppend met het

totaal.

Snelle begrippenlijst

Term	In gewone taal
canvas	Eén tekenvlak waarop je snel heel veel dingen tekent (aquarium). Je veegt het elk frame leeg en tekent opnieuw.
SVG	Losse, schaalbare vormen (cirkels, tekst, paden) — goed voor klikbare/animeerbare elementen (radar, net, talen).
context	Het "penseel" van een canvas; alle tekenopdrachten lopen erover. In de code heet het <code>context</code> .
requestAnimationFrame / raf	Vraagt de browser vóór elke schermverversing (~60×/sec) een functie te draaien. De motor van elke animatielus.
IntersectionObserver	Meldt wanneer iets in/uit beeld scrolt. Gebruikt om animaties te starten/stoppen en grafieken pas te tekenen als je ze ziet.
scaleSqrt	Een D3-schaal die via de wortel rekt, zodat de <i>oppervlakte</i> eerlijk meeschaalt (twee keer zoveel = twee keer zo veel vlak).
force-simulatie	Nep-natuurkunde van D3: elementen trekken/duwen tot ze een nette verdeling vinden (talen).
circle packing (d3.pack)	Cirkels zo compact mogelijk in een vlak passen, op grootte (net).
data-join (enter/update/exit)	D3 koppelt data aan vormen: nieuw → enter, gewijzigd → update, weg → exit. Maakt soepele overgangen.
seeded random (mulberry32)	"Toeval" met een vaste startwaarde → elke keer dezelfde reeks. Zo staan pings/posities reproduceerbaar.
rejection sampling	Net zo lang een willekeurige plek proberen tot hij aan een eis voldoet (hier: niet te dicht op een andere ping).
jitter	Een beetje vaste rommel op een waarde, zodat dingen niet té perfect/regelmatig ogen.
transition	D3 animeert een eigenschap (positie, grootte, kleur) soepel van oud naar nieuw over een tijdsduur.
ease-curve	Versnellen/afremmen tijdens een animatie i.p.v. constante snelheid (voelt natuurlijker).
pixelRatio (device pixel ratio)	Hoeveel echte pixels één CSS-pixel beslaat (retina = 2). Canvas wordt hierop afgesteld om scherp te blijven; in de code heet het <code>pixelRatio</code> .
cleanups / lifecycle	Lijst met "opruim"-functies. Bij een periodewissel of paginaverlaten worden ze gedraaid zodat niets blijft hangen.

Tot slot. Alle vier de bestanden volgen hetzelfde stramien: **data uit state** → **vorm berekenen** → **tekenen** → **interactie koppelen** → **animeren met raf** → **opruimen via cleanups**. Herken je dat patroon, dan kun je elke grafiek lezen en aanpassen.

Gegenereerd uit de actuele broncode · vervolg op de overzichts-PDF · regel-voor-regel uitleg.